

Program Example

Basic Programming

1. Write a C program to find the sum of all even numbers from 1 to n. [SO IT, Officer IT, CoB-2025]

Solution:

```
#include <stdio.h>

int main(void) {
    int n, i;
    long long sum = 0;
    scanf("%d", &n);
    for (i = 2; i <= n; i += 2) {
        sum += i;
    }
    printf("Sum = %lld\n", sum);
    return 0;
}
```

2. Write a C program to find the sum of all digits of a 5-digit number. [45th BCS]

Solution:

```
#include <stdio.h>

int main(void) {
    int n, sum = 0;
    scanf("%d", &n);
    if (n < 0)
        n = -n;
    while (n > 0) {
        sum += n % 10;
        n /= 10;
    }
    printf("Sum of digits = %d\n", sum);
    return 0;
}
```

3. Write a program to repeatedly calculate the sum of the digits of a number until a single-digit result is obtained. [Officer IT, SB-2021]

Solution:

```
#include <stdio.h>

int main(void) {
    int n, sum;
    scanf("%d", &n);
    if (n < 0)
        n = -n;
    while (n >= 10) {
        sum = 0;
        while (n > 0) {
            sum += n % 10;
            n /= 10;
        }
        n = sum;
    }
    printf("Result = %d\n", n);
    return 0;
}
```

4. Given two integers A and B, write a program to calculate their Least Common Multiple (LCM). [BB, AP-2023]

Solution:

```
#include <stdio.h>

int main() {
    int a, b, lcm;
    scanf("%d %d", &a, &b);

    if (a > b)
        lcm = a;
    else
        lcm = b;
}
```

```

while (lcm % a != 0 || lcm % b != 0) {
    lcm++;
}
printf("LCM = %d", lcm);
return 0;
}

```

5. Given two integers A and B, write a program to calculate their Greatest Common Divisor (GCD).

Solution:

```
#include <stdio.h>
```

```

int main() {
    int a, b, i;

    scanf("%d %d", &a, &b);

    if (a < b)
        i = a;
    else
        i = b;

    while (i > 0) {
        if (a % i == 0 && b % i == 0) {
            printf("GCD = %d", i);
            break;
        } else {
            i--;
        }
    }

    return 0;
}

```

6. Find the Greatest Common Divisor (GCD)

```
#include <stdio.h>

int main() {
    int a, b, i;
    scanf("%d %d", &a, &b);

    if (a < b)
        i = a;
    else
        i = b;

    while (i > 0) {
        if (a % i == 0 && b % i == 0) {
            printf("GCD = %d", i);
            break;
        } else {
            i--;
        }
    }

    return 0;
}
```

7. Find the Least Common Multiple (LCM)

```
#include <stdio.h>

int main() {
    int a, b, i;
    scanf("%d %d", &a, &b);

    if (a > b)
        i = a;
    else
        i = b;

    while (1) {
```

```

    if (i % a == 0 && i % b == 0) {
        printf("LCM = %d", i);
        break;
    } else {
        i++;
    }
}

return 0;
}

```

8. Find the GCD using a function

```
#include <stdio.h>
```

```

int gcd(int a, int b) {
    int i, small;

    if (a < b)
        small = a;
    else
        small = b;

    for (i = small; i > 0; i--) {
        if (a % i == 0 && b % i == 0)
            return i;
    }
    return 1;
}

int main() {
    int a, b;
    scanf("%d %d", &a, &b);

    printf("GCD = %d", gcd(a, b));
    return 0;
}

```

9. Write a C program to check whether a given year is a leap year.

Solution:

```
#include <stdio.h>

int main(void) {
    int year;
    scanf("%d", &year);

    if (year % 400 == 0 ||
        (year % 4 == 0 && year % 100 != 0))
        printf("Leap Year\n");
    else
        printf("Not a Leap Year\n");

    return 0;
}
```

10. Write a C program to check whether a given number is prime.

Solution:

```
#include <stdio.h>

int main(void) {
    int n, i, prime = 1;

    scanf("%d", &n);

    if (n < 2)
        prime = 0;

    for (i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            prime = 0;
            break;
        }
    }
}
```

```

}

if (prime)
    printf("Prime\n");
else
    printf("Not Prime\n");

return 0;
}

```

11. Write a program to find all prime numbers between 1 and n, where n is provided by the user.
 [SO IT, CoB-2024, JID-10185; AP, CoB-2024, JID-10198; AP, ICB-2024]

Solution:

```

#include <stdio.h>

int main(void) {
    int n, i, j, prime;

    scanf("%d", &n);

    for (i = 2; i <= n; i++) {
        prime = 1;

        for (j = 2; j * j <= i; j++) {
            if (i % j == 0) {
                prime = 0;
                break;
            }
        }

        if (prime)
            printf("%d ", i);
    }

    return 0;
}

```

Basic Programming ☞ If-Else

12. Check whether a number is even or odd

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    if (n % 2 == 0)
        printf("Even");
    else
        printf("Odd");

    return 0;
}
```

13. Find the largest of two integers

```
#include <stdio.h>

int main() {
    int a, b;
    scanf("%d %d", &a, &b);

    if (a > b)
        printf("Largest = %d", a);
    else
        printf("Largest = %d", b);

    return 0;
}
```

14. Find the largest of three integers

```
#include <stdio.h>

int main() {
    int a, b, c;
    scanf("%d %d %d", &a, &b, &c);
```

```

if (a >= b && a >= c)
    printf("%d", a);
else if (b >= a && b >= c)
    printf("%d", b);
else
    printf("%d", c);

return 0;
}

```

15. Check whether a number is positive, negative, or zero

```

#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    if (n > 0)
        printf("Positive");
    else if (n < 0)
        printf("Negative");
    else
        printf("Zero");

    return 0;
}

```

16. Check whether a year is a leap year

```

#include <stdio.h>

int main() {
    int year;
    scanf("%d", &year);

    if (year % 400 == 0 ||
        (year % 4 == 0 && year % 100 != 0))
        printf("Leap Year");
}

```

```
else  
    printf("Not Leap Year");  
  
return 0;  
}
```

17. Check whether a number is divisible by both 5 and 11

```
#include <stdio.h>  
  
int main() {  
    int n;  
    scanf("%d", &n);  
  
    if (n % 5 == 0 && n % 11 == 0)  
        printf("Divisible");  
    else  
        printf("Not Divisible");  
  
    return 0;  
}
```

18. Swap two numbers using a temporary variable

```
#include <stdio.h>  
  
int main() {  
    int a, b, temp;  
    scanf("%d %d", &a, &b);  
  
    temp = a;  
    a = b;  
    b = temp;  
  
    printf("%d %d", a, b);  
    return 0;  
}
```

19. Find the sum of the first N natural numbers

```
#include <stdio.h>

int main() {
    int n, i, sum = 0;
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
        sum += i;
    printf("Sum = %d", sum);
    return 0;
}
```

20. Find the factorial of a number

```
#include <stdio.h>

int main() {
    int n, i;
    long long fact = 1;
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
        fact *= i;

    printf("Factorial = %lld", fact);
    return 0;
}
```

21. Print the multiplication table of a number

```
#include <stdio.h>

int main() {
    int n, i;
    scanf("%d", &n);

    for (i = 1; i <= 10; i++)
        printf("%d x %d = %d\n", n, i, n * i);

    return 0;
}
```

22. Write a C program to check whether a given integer is odd or even using an if-else statement.

Solution:

```
#include <stdio.h>

int main(void) {
    int n;
    scanf("%d", &n);

    if (n % 2 == 0)
        printf("Even\n");
    else
        printf("Odd\n");

    return 0;
}
```

23. Write a program to check whether a given number is positive, negative, or zero.

Solution:

```
#include <stdio.h>

int main(void) {
    int n;
    scanf("%d", &n);

    if (n > 0)
        printf("Positive\n");
    else if (n < 0)
        printf("Negative\n");
    else
        printf("Zero\n");

    return 0;
}
```

24. Write a C program to calculate the average of three numbers.

Solution:

```
#include <stdio.h>

int main(void) {
    float a, b, c, average;

    scanf("%f %f %f", &a, &b, &c);

    average = (a + b + c) / 3.0;

    printf("Average = %.2f\n", average);
    return 0;
}
```

25. Write a C program to find the largest and smallest numbers among three given integers.

Solution:

```
#include <stdio.h>

int main(void) {
    int a, b, c, largest, smallest;

    scanf("%d %d %d", &a, &b, &c);

    largest = smallest = a;
    if (b > largest) largest = b;
    if (c > largest) largest = c;

    if (b < smallest) smallest = b;
    if (c < smallest) smallest = c;

    printf("Largest = %d\n", largest);
    printf("Smallest = %d\n", smallest);

    return 0;
}
```

26. Write a program to calculate the sum of all integers from 1 to n.

Solution:

```
#include <stdio.h>
```

```
int main(void) {
```

```
    int n, i;
```

```
    long long sum = 0;
```

```
    scanf("%d", &n);
```

```
    for (i = 1; i <= n; i++)
```

```
        sum += i;
```

```
    printf("Sum = %lld\n", sum);
```

```
    return 0;
```

```
}
```

27. Write a program to calculate the factorial of a given number n using a loop. [18th NTRCA]

Solution:

```
#include <stdio.h>
```

```
int main(void) {
```

```
    int n, i;
```

```
    unsigned long long fact = 1;
```

```
    scanf("%d", &n);
```

```
    if (n < 0) {
```

```
        printf("Not defined for negative numbers\n");
```

```
        return 0;
```

```
    }
```

```
    for (i = 1; i <= n; i++)
```

```
        fact *= i;
```

```
    printf("%d! = %llu\n", n, fact);
```

```
    return 0;
```

```
}
```

28. Find the factorial using a function

```
#include <stdio.h>

int factorial(int n) {
    int i, fact = 1;

    for (i = 1; i <= n; i++)
        fact *= i;

    return fact;
}

int main() {
    int n;
    scanf("%d", &n);

    printf("Factorial = %d", factorial(n));
    return 0;
}
```

29. Find the factorial using recursion

```
#include <stdio.h>

int fact(int n) {
    if (n <= 1)
        return 1;
    else
        return n * fact(n - 1);
}

int main() {
    int n;
    scanf("%d", &n);
    printf("Factorial = %d", fact(n));
    return 0;
}
```

30. Swap two numbers using pointers

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int a, b;
    scanf("%d %d", &a, &b);

    swap(&a, &b);

    printf("%d %d", a, b);
    return 0;
}
```

31. Store and display student information using a structure

```
#include <stdio.h>

struct Student {
    int id;
    float marks;
};

int main() {
    struct Student s;
    scanf("%d %f", &s.id, &s.marks);
    printf("ID = %d\n", s.id);
    printf("Marks = %.2f", s.marks);

    return 0;
}
```

32. Find the roots of a quadratic equationEquation: $(ax^2+bx+c=0)$

#include <stdio.h>

#include <math.h>

```

int main() {
    float a, b, c, d;
    scanf("%f %f %f", &a, &b, &c);

    d = b * b - 4 * a * c;

    if (d > 0)
        printf("Two distinct real roots");
    else if (d == 0)
        printf("Two equal real roots");
    else
        printf("Complex roots");

    return 0;
}

```

33. Write a program to calculate the area of a rectangle and a triangle using their base and height.**Solution:**

#include <stdio.h>

```

int main(void) {
    float base, height;
    scanf("%f %f", &base, &height);
    printf("Rectangle Area = %.2f\n", base * height);

    scanf("%f %f", &base, &height);
    printf("Triangle Area = %.2f\n", 0.5 * base * height);
    return 0;
}

```

34. Write a C program to find the roots of a quadratic equation $ax^2+bx+c=0$, considering distinct real roots, equal real roots, and complex roots. [16th NTRCA]

Solution:

```
#include <stdio.h>
#include <math.h>

int main(void) {
    double a, b, c, d, r1, r2, real, imag;

    scanf("%lf %lf %lf", &a, &b, &c);

    if (a == 0) {
        printf("Not a quadratic equation\n");
        return 0;
    }

    d = b * b - 4 * a * c;
    if (d > 0) {
        r1 = (-b + sqrt(d)) / (2 * a);
        r2 = (-b - sqrt(d)) / (2 * a);
        printf("Distinct roots: %.2f, %.2f\n", r1, r2);
    }
    else if (d == 0) {
        r1 = -b / (2 * a);
        printf("Equal roots: %.2f, %.2f\n", r1, r1);
    }
    else {
        real = -b / (2 * a);
        imag = sqrt(-d) / (2 * a);
        printf("Complex roots: %.2f + %.2fi, %.2f - %.2fi\n",
            real, imag, real, imag);
    }

    return 0;
}
```

35. Write a program to calculate the sum of the series $e^x = 1 + x/1! + x^2/2! + x^3/3! + \dots$. [ADA, CoB-2024, JID-10200]

Solution:

```
#include <stdio.h>

int main(void) {
    double x, sum = 1.0, term = 1.0;
    int n, i;
    scanf("%lf %d", &x, &n);

    for (i = 1; i < n; i++) {
        term = term * x / i;
        sum += term;
    }
    printf("Sum = %.6f\n", sum);
    return 0;
}
```

36. Write a C++ program to calculate the sum of the series $1+2+4+7+11+\dots+N$. [AP, CoB-2024, JID-10198]

Solution (C++):

```
#include <iostream>
using namespace std;

int main() {
    int N, term = 1, difference = 1;
    long long sum = 0;
    cin >> N;
    while (term <= N) {
        sum += term;
        difference++;
        term += difference;
    }
    cout << "Sum = " << sum << endl;
    return 0;
}
```

37. Write a program to calculate an employee's total weekly pay using hourly wage, regular hours, and overtime hours, where overtime pay = overtime hours $\times$ 1.5 $\times$ hourly wage. [Officer IT, SB & JB-2020]

Solution:

```
#include <stdio.h>

int main(void) {
    float wage, regularHours, overtimeHours;
    float regularPay, overtimePay, totalPay;
    scanf("%f %f %f", &wage, &regularHours, &overtimeHours);
    regularPay = wage * regularHours;
    overtimePay = overtimeHours * 1.5 * wage;
    totalPay = regularPay + overtimePay;

    printf("Regular Pay = %.2f\n", regularPay);
    printf("Overtime Pay = %.2f\n", overtimePay);
    printf("Total Weekly Pay = %.2f\n", totalPay);

    return 0;
}
```

38. Write a program that takes two integers as input and displays their bitwise AND, OR, and XOR results.

Solution:

```
#include <stdio.h>

int main(void) {
    unsigned int a, b;

    scanf("%u %u", &a, &b);

    printf("AND = %u\n", a & b);
    printf("OR  = %u\n", a | b);
    printf("XOR = %u\n", a ^ b);

    return 0;
}
```

Looping & Control Statements

39. Write a program using a for loop to print all integers from 1 to 100.

Solution:

```
#include <stdio.h>

int main(void) {
    int i;
    for (i = 1; i <= 100; i++)
        printf("%d ", i);
    return 0;
}
```

40. Write a program using a while loop to print all integers from 1 to 100.

Solution:

```
#include <stdio.h>

int main(void) {
    int i = 1;
    while (i <= 100) {
        printf("%d ", i);
        i++;
    }

    return 0;
}
```

41. Write a program using a do-while loop to print all integers from 1 to 100.

Solution:

```
#include <stdio.h>

int main(void) {
    int i = 1;
    do {
        printf("%d ", i);
        i++;
    } while (i <= 100);
    return 0;
}
```

Arrays**42. Find the sum of array elements**

```
#include <stdio.h>

int main() {
    int a[100], n, i, sum = 0;
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        sum += a[i];
    }
    printf("Sum = %d", sum);
    return 0;
}
```

43. Find the largest element in an array

```
#include <stdio.h>

int main() {
    int a[100], n, i, max;
    scanf("%d", &n);

    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    max = a[0];

    for (i = 1; i < n; i++) {
        if (a[i] > max)
            max = a[i];
    }

    printf("Maximum = %d", max);
    return 0;
}
```

44. Find the smallest element in an array

```
#include <stdio.h>

int main() {
    int a[100], n, i, min;
    scanf("%d", &n);

    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    min = a[0];

    for (i = 1; i < n; i++) {
        if (a[i] < min)
            min = a[i];
    }
    printf("Minimum = %d", min);
    return 0;
}
```

45. Calculate the average of array elements

```
#include <stdio.h>

int main() {
    int a[100], n, i, sum = 0;
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        sum += a[i];
    }

    printf("Average = %.2f", (float)sum / n);
    return 0;
}
```

46. Reverse an array

```
#include <stdio.h>

int main() {
    int a[100], n, i, temp;
    scanf("%d", &n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    for (i = 0; i < n / 2; i++) {
        temp = a[i];
        a[i] = a[n - 1 - i];
        a[n - 1 - i] = temp;
    }
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    return 0;
}
```

47. Write a C program to calculate the average of the elements of an array. [16th NTRCA]

Solution:

```
#include <stdio.h>

int main(void) {
    int a[100], n, i;
    float sum = 0, average;
    scanf("%d", &n);
    if (n <= 0 || n > 100) {
        printf("Invalid array size\n");
        return 0;
    }
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        sum += a[i];
    }
    average = sum / n;
    printf("Average = %.2f\n", average);
    return 0; }
```

48. Write a C program to calculate both the sum and the product of four elements in an array.

Solution:

```
#include <stdio.h>

int main(void) {
    int a[4], i, sum = 0;
    long long product = 1;

    for (i = 0; i < 4; i++) {
        scanf("%d", &a[i]);
        sum += a[i];
        product *= a[i];
    }

    printf("Sum = %d\n", sum);
    printf("Product = %lld\n", product);

    return 0;
}
```

49. Write a C program that accepts the size and elements of an array and prints the elements in reverse order. [17th NTRCA]

Solution:

```
#include <stdio.h>

int main(void) {
    int a[100], n, i;
    scanf("%d", &n);
    if (n <= 0 || n > 100)
        return 0;
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    for (i = n - 1; i >= 0; i--)
        printf("%d ", a[i]);
    return 0;
}
```

50. Write a function that receives an array of integers as a parameter and prints all numbers divisible by 3. [Officer IT, CoB-2024, JID-10184; Officer IT, CoB-2025]

Solution:

```
#include <stdio.h>

void divisibleBy3(int a[], int n) {
    int i;

    for (i = 0; i < n; i++) {
        if (a[i] % 3 == 0)
            printf("%d ", a[i]);
    }
}

int main(void) {
    int a[] = {3, 7, 9, 10, 12, 14};
    int n = sizeof(a) / sizeof(a[0]);
    divisibleBy3(a, n);
    return 0;
}
```

Output: 3 9 12

51. Write a function to find the smallest element in an array. [AP, CoB-2023]

Solution:

```
#include <stdio.h>

int smallestElement(int a[], int n) {
    int i, smallest = a[0];
    for (i = 1; i < n; i++) {
        if (a[i] < smallest)
            smallest = a[i];
    }
    return smallest;
}

int main(void) {
    int a[] = {12, 5, 8, -2, 17};
```

```

int n = sizeof(a) / sizeof(a[0]);
printf("Smallest = %d\n",
      smallestElement(a, n));

return 0;
}

```

Output: Smallest = -2

52. Write a program to print the common elements between two arrays and count the total number of common elements. Assume that each array contains distinct elements. [AD-ICT, BB-2025]

Solution:

```
#include <stdio.h>
```

```

int main(void) {
    int a[] = {1, 2, 3, 4, 5};
    int b[] = {3, 4, 5, 6, 7};
    int n = 5, m = 5;
    int i, j, count = 0;

    printf("Common elements: ");
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            if (a[i] == b[j]) {
                printf("%d ", a[i]);
                count++;
                break;
            }
        }
    }

    printf("\nTotal common elements = %d\n", count);
    return 0;
}

```

Output: Common elements: 3 4 5

Total common elements = 3

53. An array of size 10 contains all integers from 1 to 10, but one number has been replaced by zero. Write a C/C++ program using a function to identify and restore the missing number. [Ministry of Food, Network/Website Manager (ICT)–2025]

Solution:

```
#include <stdio.h>

int restoreMissing(int a[], int n) {
    int expected = n * (n + 1) / 2;
    int actual = 0;
    int i, missing, zeroIndex = -1;

    for (i = 0; i < n; i++) {
        actual += a[i];

        if (a[i] == 0)
            zeroIndex = i;
    }
    missing = expected - actual;

    if (zeroIndex != -1)
        a[zeroIndex] = missing;

    return missing;
}

int main(void) {
    int a[10] = {1, 2, 3, 4, 0, 6, 7, 8, 9, 10};
    int i, missing;
    missing = restoreMissing(a, 10);
    printf("Missing number = %d\n", missing);
    printf("Restored array: ");

    for (i = 0; i < 10; i++)
        printf("%d ", a[i]);

    return 0;
}
```

54. Write a program that takes an array containing positive and negative numbers and rearranges it so that positive numbers appear before negative numbers. [MTO, DBBL-2024]

Solution:

```
#include <stdio.h>

int main(void) {
    int a[] = {-2, 5, -7, 4, 0, -1, 8};
    int n = sizeof(a) / sizeof(a[0]);
    int i, next = 0, temp;

    for (i = 0; i < n; i++) {
        if (a[i] >= 0) {
            temp = a[i];
            a[i] = a[next];
            a[next] = temp;
            next++;
        }
    }

    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

Mathematical Algorithms

55. Write a C program to generate the first 10 Fibonacci numbers. [17th NTRCA]

Solution:

```
#include <stdio.h>

int main(void) {
    int a = 0, b = 1, next, i;

    for (i = 0; i < 10; i++) {
        printf("%d ", a);
```

```

    next = a + b;
    a = b;
    b = next;
}
return 0;
}

```

Output: 0 1 1 2 3 5 8 13 21 34

56. Write a C program to generate the Fibonacci series for n terms, where n is provided by the user.

[MTO, DBBL-2024; AME, BPSC-2024]

Solution:

```

#include <stdio.h>
int main(void) {
    int n, i;
    long long a = 0, b = 1, next;

    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("%lld ", a);

        next = a + b;
        a = b;
        b = next;
    }
    return 0;
}

```

57. Given the recursive function $F(n)=F(n-2)+F(n-1)$, determine the output when main() calls F(5).

[Officer IT, CoB-2023, JID-10152; AP, CoB-2024, JID-10198]

Solution:

```

#include <stdio.h>
int F(int n) {
    if (n <= 1)
        return n;
}

```

```

    return F(n - 2) + F(n - 1);
}

```

```

int main(void) {
    printf("%d\n", F(5));
    return 0;
}

```

Output: 5

58. Write a recursive algorithm or program to calculate the factorial of a number.

[Officer IT, SB & JB-2020]

Solution:

```

#include <stdio.h>

unsigned long long factorial(int n) {
    if (n <= 1)
        return 1;
    return n * factorial(n - 1);
}

int main(void) {
    int n;
    scanf("%d", &n);

    if (n < 0)
        printf("Factorial is not defined\n");
    else
        printf("Factorial = %llu\n", factorial(n));

    return 0;
}

```

59. Write a recursive function that takes a string as a parameter and returns a Boolean value indicating whether the string is a palindrome. [SO IT, CoB-2022]

Solution:

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

bool isPalindrome(char s[], int left, int right) {
    if (left >= right)
        return true;

    if (s[left] != s[right])
        return false;
    return isPalindrome(s, left + 1, right - 1);
}

int main(void) {
    char s[100];
    scanf("%99s", s);
    if (isPalindrome(s, 0, (int)strlen(s) - 1))
        printf("True\n");
    else
        printf("False\n");
    return 0;
}
```

60. Find the output of the given recursive function containing recursive calls and printing statements. [AP, CoB-2024, JID-10198]

Solutions:

```
#include <stdio.h>

int fun(int i) {
    if (i <= 1)
        return 1;
    return i + fun(i - 1);
}
```

```
int main(void) {
    printf("%d\n", fun(4));
    return 0;
}
```

Output: 10

Matrix

61. Write a program to calculate the sum of each row and each column of an $m \times n$ matrix, where m and n are provided by the user. [SO IT, CoB-2024, JID-10185; SO IT, CoB-2024]

Solution:

```
#include <stdio.h>

int main(void) {
    int a[20][20], m, n;
    int i, j, sum;
    scanf("%d %d", &m, &n);
    if (m < 1 || m > 20 || n < 1 || n > 20)
        return 0;
    for (i = 0; i < m; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
        }
    }

    for (i = 0; i < m; i++) {
        sum = 0;
        for (j = 0; j < n; j++)
            sum += a[i][j];
        printf("Row %d Sum = %d\n", i + 1, sum);
    }

    for (j = 0; j < n; j++) {
        sum = 0;
        for (i = 0; i < m; i++)
            sum += a[i][j];
    }
}
```

```

    printf("Column %d Sum = %d\n", j + 1, sum);
}
return 0;
}

```

62. Write a program that takes two matrices A and B as input, handles different matrix dimensions, and calculates: [Combined Bank, SO IT-2023]

- (a) the product matrix C,
- (b) the average of the elements of A and B separately, and
- (c) the maximum element of C.

Solution:

```

#include <stdio.h>

int main(void) {
    int A[10][10], B[10][10], C[10][10];
    int r1, c1, r2, c2;
    int i, j, k, maxC;
    long long sumA = 0, sumB = 0;
    int maxSet = 0;

    /* Input matrix A */
    scanf("%d %d", &r1, &c1);

    if (r1 < 1 || r1 > 10 || c1 < 1 || c1 > 10)
        return 0;

    for (i = 0; i < r1; i++) {
        for (j = 0; j < c1; j++) {
            scanf("%d", &A[i][j]);
            sumA += A[i][j];
        }
    }

    /* Input matrix B */
    scanf("%d %d", &r2, &c2);

```

```

if (r2 < 1 || r2 > 10 || c2 < 1 || c2 > 10)
    return 0;

```

```

for (i = 0; i < r2; i++) {
    for (j = 0; j < c2; j++) {
        scanf("%d", &B[i][j]);
        sumB += B[i][j];
    }
}

```

```

printf("Average of A = %.2f\n",
    (double)sumA / (r1 * c1));

```

```

printf("Average of B = %.2f\n",
    (double)sumB / (r2 * c2));

```

```

if (c1 != r2) {
    printf("Matrix multiplication is not possible\n");
    return 0;
}

```

```

/* Calculate product matrix C */

```

```

for (i = 0; i < r1; i++) {
    for (j = 0; j < c2; j++) {
        C[i][j] = 0;

        for (k = 0; k < c1; k++)
            C[i][j] += A[i][k] * B[k][j];
    }
}

```

```

if (!maxSet || C[i][j] > maxC) {
    maxC = C[i][j];
    maxSet = 1;
}

```

```

}
}

```

```

printf("Product Matrix C:\n");

for (i = 0; i < r1; i++) {
    for (j = 0; j < c2; j++)
        printf("%d ", C[i][j]);

    printf("\n");
}

printf("Maximum element of C = %d\n", maxC);

return 0;
}

```

63. Add two matrices

```

#include <stdio.h>

int main() {
    int a[10][10], b[10][10], i, j, r, c;
    scanf("%d %d", &r, &c);

    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            scanf("%d", &a[i][j]);

    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            scanf("%d", &b[i][j]);

    for (i = 0; i < r; i++) {
        for (j = 0; j < c; j++)
            printf("%d ", a[i][j] + b[i][j]);
        printf("\n");
    }

    return 0;
}

```

64. Find the transpose of a matrix

```
#include <stdio.h>

int main() {
    int a[10][10], r, c, i, j;
    scanf("%d %d", &r, &c);

    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            scanf("%d", &a[i][j]);

    for (j = 0; j < c; j++) {
        for (i = 0; i < r; i++)
            printf("%d ", a[i][j]);
        printf("\n");
    }

    return 0;
}
```

65. Find the sum of the main diagonal of a square matrix

```
#include <stdio.h>

int main() {
    int a[10][10], n, i, j, sum = 0;
    scanf("%d", &n);

    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
            if (i == j)
                sum += a[i][j];
        }

    printf("Diagonal Sum = %d", sum);
    return 0;
}
```

Date Validation & Pattern Printing

66. Write a program that takes a date, month, and year as input, checks whether the date is valid, and displays “Valid Date” or “Invalid Date”. Explain how leap years are handled. [MTO, DBBL-2024]

Solution:

```
#include <stdio.h>

int main(void) {
    int d, m, y;
    int monthDays[] = {
        31, 28, 31, 30, 31, 30,
        31, 31, 30, 31, 30, 31
    };

    scanf("%d %d %d", &d, &m, &y);
    if (y < 1 || m < 1 || m > 12 || d < 1) {
        printf("Invalid Date\n");
        return 0;
    }

    if (y % 400 == 0 ||
        (y % 4 == 0 && y % 100 != 0)) {
        monthDays[1] = 29;
    }

    if (d <= monthDays[m - 1])
        printf("Valid Date\n");
    else
        printf("Invalid Date\n");
    return 0;
}
```

67. Write a C/Java program to print Floyd's Triangle for n rows, where n is provided by the user.

[AP, CoB-2024, JID-10198;]

Solution:

```
#include <stdio.h>

int main(void) {
    int n, i, j, value = 1;

    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i; j++) {
            printf("%d ", value);
            value++;
        }
        printf("\n");
    }
    return 0;
}
```

Input: 4

Output:

```
1
2 3
4 5 6
7 8 9 10
```

Operators & Expression Evaluation

68. Consider the following C-like code. [46th BCS]

```
int a = 5, b = 18, c = 27;
a = b++ + ++c;
b = a++ - c--;
c = a++ + b--;
```

Determine the final values of a, b, and c, and explain the effect of the increment and decrement operators.

Solution:

Given:

int a = 5, b = 18, c = 27;

a = b++ + ++c;

b = a++ - c--;

c = a++ + b--;

Step 1: a = b++ + ++c;

- b++ uses the current value of b (18), then increments b to 19.
- ++c increments c to 28 before using its value.

Therefore, [a = 18 + 28 = 46]

Current values: a = 46, b = 19, c = 28

Step 2: b = a++ - c--;

- a++ uses 46, then increments a to 47.
- c-- uses 28, then decrements c to 27.

Therefore, [b = 46 - 28 = 18]

Current values: a = 47, b = 18, c = 27

Step 3: c = a++ + b--;

- a++ uses 47, then increments a to 48.
- b-- uses 18, then decrements b to 17.

Therefore, [c = 47 + 18 = 65]

Final Answer

Variable	Final Value
a	48
b	17
c	65

Effect of Increment and Decrement Operators

Operator	Name	Effect
++a	Pre-increment	Increments the value before using it.
a++	Post-increment	Uses the current value, then increments it.
--a	Pre-decrement	Decrements the value before using it.
a--	Post-decrement	Uses the current value, then decrements it.

Database

69. Write a program using a database that allows a client to log in to a page using an ID and password. [ADA, CoB-2024, JID-10200]

Solutions:**Step 1: Create the database table**

```
CREATE TABLE users (
    user_id VARCHAR(50) PRIMARY KEY,
    password_hash VARCHAR(255) NOT NULL
);
```

Step 2: Login query

```
SELECT user_id, password_hash
FROM users
WHERE user_id = ?;
```

এখানে ? হলো parameter placeholder, যা prepared statement-এর মাধ্যমে ব্যবহার করতে হবে।

Step 3: Login algorithm

```
START
INPUT user_id
INPUT password
SEARCH user_id in database

IF user_id exists THEN
    VERIFY password against stored password hash

    IF password is correct THEN
        DISPLAY "Login Successful"
    ELSE
        DISPLAY "Invalid ID or Password"
    END IF

ELSE
    DISPLAY "Invalid ID or Password"
END IF

STOP
```

Others

70. Write a program in an object-oriented language to represent a Bank Account. The class should have private data members for the account holder's name, account number, and balance, and public methods for deposit(), withdraw() and display(). The withdrawal method must ensure that sufficient balance is available. Demonstrate encapsulation. [Officer IT, CoB-2025;]

Solution (C++):

```
#include <iostream>
#include <string>
using namespace std;
```

```
class BankAccount {
```

```
private:
```

```
    string holderName;
    string accountNumber;
    double balance;
```

```
public:
```

```
    BankAccount(string name, string number, double initialBalance) {
```

```
        holderName = name;
        accountNumber = number;
        balance = (initialBalance >= 0) ? initialBalance : 0;
```

```
    }
```

```
    void deposit(double amount) {
```

```
        if (amount > 0) {
            balance += amount;
            cout << "Deposit successful\n";
        } else {
            cout << "Invalid deposit amount\n";
        }
    }
```

```
    void withdraw(double amount) {
```

```
        if (amount <= 0) {
```

```
        cout << "Invalid withdrawal amount\n";
    } else if (amount > balance) {
        cout << "Insufficient balance\n";
    } else {
        balance -= amount;
        cout << "Withdrawal successful\n";
    }
}

void display() {
    cout << "Account Holder: " << holderName << endl;
    cout << "Account Number: " << accountNumber << endl;
    cout << "Balance: " << balance << endl;
}

};

int main() {
    BankAccount account("Rahim", "100200300", 5000);

    account.deposit(1000);
    account.withdraw(1500);
    account.display();

    return 0;
}
```

Output:

Deposit successful

Withdrawal successful

Account Holder: Rahim

Account Number: 100200300

Balance: 4500

Sorting Algorithms

45. Insertion sort is a simple sorting algorithm. Write a program to sort some given numbers using insertion sort algorithm. [Officer IT, SB & JB - 2020]

Solutions:

```
#include <stdio.h>

int main(void) {
    int a[] = {9, 5, 1, 4, 3};
    int n = sizeof(a) / sizeof(a[0]);
    int i, j, key;

    for (i = 1; i < n; i++) {
        key = a[i];
        j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }

    printf("Sorted numbers: ");

    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }

    return 0;
}
```

71. Search for an element using linear search

```
#include <stdio.h>

int main() {
    int a[100], n, i, key, found = 0;
    scanf("%d", &n);
```

```

for (i = 0; i < n; i++)
    scanf("%d", &a[i]);

scanf("%d", &key);

for (i = 0; i < n; i++) {
    if (a[i] == key) {
        printf("Found at position %d", i + 1);
        found = 1;
        break;
    }
}

if (!found)
    printf("Not Found");

return 0;
}

```

72. Sort array elements in ascending order using bubble sort

```

#include <stdio.h>

int main() {
    int a[100], n, i, j, temp;
    scanf("%d", &n);

    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}

```

```

    }
}

for (i = 0; i < n; i++)
    printf("%d ", a[i]);

return 0;
}

```

Number-Based Programming

73. Check whether a number is prime

```

#include <stdio.h>

int main() {
    int n, i, prime = 1;
    scanf("%d", &n);

    if (n < 2)
        prime = 0;
    else {
        for (i = 2; i * i <= n; i++) {
            if (n % i == 0) {
                prime = 0;
                break;
            }
        }
    }

    if (prime)
        printf("Prime");
    else
        printf("Not Prime");

    return 0;
}

```

74. Print prime numbers from 1 to N

```
#include <stdio.h>

int main() {
    int n, i, j, prime;
    scanf("%d", &n);

    for (i = 2; i <= n; i++) {
        prime = 1;
        for (j = 2; j * j <= i; j++) {
            if (i % j == 0) {
                prime = 0;
                break;
            }
        }
        if (prime)
            printf("%d ", i);
    }

    return 0;
}
```

75. Find the sum of the digits of a number

```
#include <stdio.h>

int main() {
    int n, sum = 0;
    scanf("%d", &n);

    if (n < 0) n = -n;

    while (n > 0) {
        sum += n % 10;
        n /= 10;
    }

    printf("Sum = %d", sum);
}
```

```
return 0;  
}
```

76. Reverse a number

```
#include <stdio.h>  
int main() {  
    int n, rem, rev = 0;  
    scanf("%d", &n);  
  
    while (n != 0) {  
        rem = n % 10;  
        rev = rev * 10 + rem;  
        n /= 10;  
    }  
  
    printf("Reverse = %d", rev);  
    return 0;  
}
```

77. Check whether a number is a palindrome

```
#include <stdio.h>  
int main() {  
    int n, temp, rem, rev = 0;  
    scanf("%d", &n);  
    temp = n;  
  
    while (n > 0) {  
        rem = n % 10;  
        rev = rev * 10 + rem;  
        n /= 10;  
    }  
  
    if (temp == rev)
```

```

    printf("Palindrome");
else
    printf("Not Palindrome");
return 0;
}

```

78. Check whether a number is an Armstrong number

Example: $153 = 1^3 + 5^3 + 3^3$

```

#include <stdio.h>

int main() {
    int n, temp, rem, sum = 0;
    scanf("%d", &n);
    temp = n;

    while (n > 0) {
        rem = n % 10;
        sum += rem * rem * rem;
        n /= 10;
    }

    if (temp == sum)
        printf("Armstrong");
    else
        printf("Not Armstrong");

    return 0;
}

```

79. Count the digits of a number

```

#include <stdio.h>

int main() {
    int n, count = 0;
    scanf("%d", &n);

    if (n < 0) n = -n;
}

```

```

do {
    count++;
    n /= 10;
} while (n != 0);

printf("Digits = %d", count);
return 0;
}

```

80. Find the power of a number without using pow()

```

#include <stdio.h>

int main() {
    int a, b, i, result = 1;
    scanf("%d %d", &a, &b);

    for (i = 1; i <= b; i++)
        result *= a;

    printf("Result = %d", result);
    return 0;
}

```

81. Print the Fibonacci series

```

#include <stdio.h>

int main() {
    int n, i, a = 0, b = 1, c;
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        printf("%d ", a);
        c = a + b;
        a = b;
        b = c;
    }
}

```

```
return 0;  
}
```

82. Check whether a number is a perfect number

Example: $6 = 1 + 2 + 3$

```
#include <stdio.h>  
  
int main() {  
    int n, i, sum = 0;  
    scanf("%d", &n);  
  
    for (i = 1; i < n; i++) {  
        if (n % i == 0)  
            sum += i;  
    }  
  
    if (sum == n && n > 0)  
        printf("Perfect Number");  
    else  
        printf("Not Perfect");  
  
    return 0;  
}
```

83. Find the largest digit in a number

```
#include <stdio.h>  
  
int main() {  
    int n, digit, largest = 0;  
    scanf("%d", &n);  
  
    if (n < 0) n = -n;  
  
    do {  
        digit = n % 10;
```

```

    if (digit > largest)
        largest = digit;
    n /= 10;
} while (n > 0);

printf("Largest Digit = %d", largest);
return 0;
}

```

84. Find the smallest digit in a number

```

#include <stdio.h>

int main() {
    int n, digit, smallest = 9;
    scanf("%d", &n);

    if (n < 0) n = -n;

    do {
        digit = n % 10;
        if (digit < smallest)
            smallest = digit;
        n /= 10;
    } while (n > 0);

    printf("Smallest Digit = %d", smallest);
    return 0;
}

```

85. Check whether a number is divisible by the sum of its digits

```

#include <stdio.h>

int main() {
    int n, temp, sum = 0;
    scanf("%d", &n);
    temp = n;

```

```

while (temp > 0) {
    sum += temp % 10;
    temp /= 10;
}

if (sum != 0 && n % sum == 0)
    printf("Divisible");
else
    printf("Not Divisible");

return 0;
}

```

86. Sort array elements in descending order

```

#include <stdio.h>

int main() {
    int a[100], n, i, j, temp;
    scanf("%d", &n);

    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - 1 - i; j++) {
            if (a[j] < a[j + 1]) {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }

    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
}

```

```

return 0;
}

```

87. Find the second-largest distinct element in an array

```

#include <stdio.h>
#include <limits.h>
int main() {
    int a[100], n, i, max = INT_MIN;
    int second = INT_MIN;
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);

        if (a[i] > max) {
            second = max;
            max = a[i];
        } else if (a[i] > second && a[i] < max) {
            second = a[i];
        }
    }

    if (second == INT_MIN)
        printf("No second-largest distinct element");
    else
        printf("Second Largest = %d", second);

    return 0;
}

```

88. Count even and odd elements in an array

```

#include <stdio.h>
int main() {
    int a[100], n, i, even = 0, odd = 0;

```

```

scanf("%d", &n);

for (i = 0; i < n; i++) {
    scanf("%d", &a[i]);

    if (a[i] % 2 == 0)
        even++;
    else
        odd++;
}

printf("Even = %d\nOdd = %d", even, odd);
return 0;
}

```

String Programming

89. Find the length of a string without using strlen()

```

#include <stdio.h>

int main() {
    char s[100];
    int i = 0;

    fgets(s, sizeof(s), stdin);

    while (s[i] != '\0' && s[i] != '\n')
        i++;

    printf("Length = %d", i);
    return 0;
}

```

90. Reverse a string

```

#include <stdio.h>
#include <string.h>

int main() {

```

```
char s[100], temp;
int i, len;

scanf("%99s", s);
len = strlen(s);

for (i = 0; i < len / 2; i++) {
    temp = s[i];
    s[i] = s[len - 1 - i];
    s[len - 1 - i] = temp;
}

printf("%s", s);
return 0;
}
```

91. Check whether a string is a palindrome

```
#include <stdio.h>
#include <string.h>
int main() {
    char s[100];
    int i, len, palindrome = 1;

    scanf("%99s", s);
    len = strlen(s);

    for (i = 0; i < len / 2; i++) {
        if (s[i] != s[len - 1 - i]) {
            palindrome = 0;
            break;
        }
    }

    if (palindrome)
        printf("Palindrome");
}
```

```

else
    printf("Not Palindrome");

return 0;
}

```

92. Count vowels in a string

```

#include <stdio.h>

int main() {
    char s[100];
    int i, count = 0;

    scanf("%99s", s);

    for (i = 0; s[i] != '\0'; i++) {
        if (s[i] == 'a' || s[i] == 'e' ||
            s[i] == 'i' || s[i] == 'o' ||
            s[i] == 'u' || s[i] == 'A' ||
            s[i] == 'E' || s[i] == 'I' ||
            s[i] == 'O' || s[i] == 'U')
            count++;
    }

    printf("Vowels = %d", count);
    return 0;
}

```

93. Copy one string to another without using strcpy()

```

#include <stdio.h>

int main() {
    char s1[100], s2[100];
    int i = 0;

    scanf("%99s", s1);

```

```
while (s1[i] != '\0') {  
    s2[i] = s1[i];  
    i++;  
}  
  
s2[i] = '\0';  
printf("%s", s2);  
  
return 0;  
}
```

94. Compare two strings without using strcmp()

```
#include <stdio.h>  
  
int main() {  
    char a[100], b[100];  
    int i = 0;  
  
    scanf("%99s %99s", a, b);  
  
    while (a[i] == b[i] && a[i] != '\0')  
        i++;  
  
    if (a[i] == b[i])  
        printf("Equal");  
    else  
        printf("Not Equal");  
  
    return 0;  
}
```

OOP

1. Write a program using Object, Class, Inheritance, Polymorphism, Encapsulation, overridden

ANS:

```
#include <iostream>
using namespace std;
// এটা Base class -
class Shape {

// ICT WIZARD নোটঃ Shape এটা বেইস বা মূল ক্লাস যেটাকে আমরা ইনহেরিট করব
public:
    virtual int getArea() {
        return 0;
    } // এখানে getArea হচ্ছে Shape ক্লাসের মেম্বার ফাংশন যা আমরা পরবর্তি ড্রাইভ ক্লাসগুলোতে অভাররাইড করে
    পলিমরফিজম এচিভ করব।

};

// এটা Derived class
class Rectangle: public Shape {

// এখানে Rectangle নামে মূল ক্লাস Shape কে ইনহেরিট করা হইছে
public:
    Rectangle(int a = 0, int b = 0) {
        width = a;
        height = b;
    }
    int getArea() {
        return (width*height);
    }
private:
    int width;
    int height;
};
```

```
int main() {  
    Shape *shape; // এখানে Shape ক্লাসের পয়েন্টার ক্রিয়েট করা হইছে।  
    Rectangle rec(10,7); // rec নামে অবজেক্ট ক্রিয়েট করা হইছে Rectangle ক্লাসের  
    shape = &rec; // rec এর এড্রেসকে shape এর মধ্যে রাখা হয়েছে।  
    cout << "Rectangle area: " << shape->getArea() << endl;  
    // এখানে getArea ওভাররিডেনের মাধ্যমে পলিমরফিজম হয়েছে এটিভ হইছে।  
  
    return 0;  
}
```

